

# Advanced Database Systems - MIT 8107

## Redis Key-Value Store Lab Guide

**Duration: 30-60 minutes**

**Data Model: Key-Value Store**

### Table of Contents

1. Introduction
2. Setup Instructions
3. Basic Operations
4. Applied Scenario: Real-Time Session Management System
5. Advanced Operations
6. Group Contribution Summary

## 1. Introduction

### What is a Key-Value Database?

A key-value database is the simplest type of NoSQL database that stores data as a collection of key-value pairs. Each key is unique and acts as an identifier to retrieve its associated value. Think of it like a dictionary or hash map.

### Key Characteristics:

- ☐ Simple structure: Key → Value
- ☐ Fast lookups:  $O(1)$  time complexity
- ☐ Flexible values: Can store strings, numbers, lists, sets, hashes, etc.
- ☐ High performance: Optimized for speed and scalability

### Why Redis?

Redis (Remote Dictionary Server) is an open-source, in-memory key-value store known for: extremely fast read/write operations (sub-millisecond latency), rich data types (strings, hashes, lists, sets, sorted sets), built-in persistence options, wide industry adoption (Twitter, GitHub, Stack Overflow)

## When to Use Key-Value Stores?

Key-value DBs are ideal for session management and caching, real-time analytics and leaderboards, rate limiting and API throttling, shopping carts, User preferences and profiles and Message queues.

On the other hand, they are not a best suite for complex queries with multiple conditions, relationships between entities, data requiring frequent aggregations.

## 2. Setup Instructions:

### Prerequisites

1. VS Code: [link](#)
2. Git and Git Bash: [link](#)
3. GitHub Desktop: [link](#)
4. WSL (for Windows OS):[link](#)
5. Docker: [link](#)
6. VS Code extensions:
7. Docker and YAML

### Step 1: Pull Redis Docker Image

Enter the following command to pull the redis docker image

```
docker pull redis:7.2-alpine
```

### Step 2: Run Redis Container

Enter the following command to start Redis container with persistent storage:

```
docker run -d \  
  --name redis-lab \  
  -p 6379:6379 \  
  -v redis-data:/data \  
  redis:7.2-alpine redis-server --appendonly yes
```

### Command Breakdown:

- `-d`: Run in detached mode (background)
- `--name redis-lab`: Name the container for easy reference
- `-p 6379:6379`: Map port 6379 (host:container)
- `-v redis-data:/data`: Create volume for data persistence
- `--appendonly yes`: Enable AOF (Append Only File) persistence

### Step 3: Verify Installation

Enter the following command to check if container is running

```
docker ps
```

Expected Output: {image}

### Step 4: Connect to Redis CLI

Enter the following command to connect to Redis command-line interface

```
docker exec -it redis-lab redis-cli
```

You should see the Redis prompt:

```
127.0.0.1:6379>
```

### Step 5: Test Connection

Inside Redis CLI, test with PING command

```
127.0.0.1:6379> PING
```

Expected Output: {image}

## 3. Basic Operations (CRUD)

### Understanding Redis Data Types

Before we dive into CRUD operations, let's understand Redis's core data types:

1. String - Simple key-value pairs
2. Hash - Key with multiple field-value pairs (like objects)
3. List - Ordered collection of strings
4. Set - Unordered collection of unique strings
5. Sorted Set - Set ordered by score

### CREATE Operations

#### A. String Operations

Enter the following command to set a simple key-value pair

```
SET user:1001:name "Alice Johnson"
```

Enter the following command to set with expiration (EX = seconds, PX = milliseconds)

```
SET session:abc123 "user_data" EX 3600
```

Enter the following command to set only if key doesn't exist (NX = Not eXists)

```
SET user:1001:created_at "2025-11-28" NX
```

Enter the following command to set multiple key-value pairs at once

```
MSET user:1001:email "alice@email.com" user:1001:age "28"
user:1001:country "Kenya"
```

Expected Output: {image}

## **B. Hash Operations (Storing Objects)**

Enter the following command to create a hash with multiple fields

```
HSET user:1002 name "Bob Mwangi" email "bob@email.com" age 32
country "Kenya"
```

Enter the following command to add individual fields to hash

```
HSET user:1002 role "developer"
HSET user:1002 department "Engineering"
```

Expected Output: {image}

## **C. List Operations**

Enter the following commands to create a list (push from left/right)

```
RPUSH user:1001:login_history "2025-11-28 09:00:00"
RPUSH user:1001:login_history "2025-11-28 14:30:00"
LPUSH user:1001:notifications "Welcome to the platform!"
```

## **D. Set Operations**

Enter the following command to add members to a set

```
SADD user:1001:skills "Python" "Java" "Docker" "Redis"
SADD user:1002:skills "JavaScript" "Python" "React"
```

## **READ Operations**

### **A. String Operations**

Enter the following command to get a single value

```
GET user:1001:name
```

Enter the following command to get multiple values at once

```
MGET user:1001:name user:1001:email user:1001:age
```

Enter the following command to check if key exists

```
EXISTS user:1001:name
```

Enter the following command to get all keys matching a pattern (use carefully in production!)

```
KEYS user:1001:*
```

Expected Output: {image}

## B. Hash Operations

Enter the following command to get all fields and values from a hash

**HGETALL** *user:1002*

Enter the following command to get specific field from hash

**HGET** *user:1002 name*

Enter the following command to get multiple fields from hash

**HMGET** *user:1002 name email role*

Enter the following command to get all field names

**HKEYS** *user:1002*

Enter the following command to get all values

**HVALS** *user:1002*

Enter the following command to check if field exists

**HEXISTS** *user:1002 age*

Expected Output for HGETALL: {image}

## C. List Operations

Enter the following command to get all elements from a list

**LRANGE** *user:1001:login\_history 0 -1*

Enter the following command to get list length

**LLEN** *user:1001:login\_history*

Enter the following command to get element at specific index

**LINDEX** *user:1001:login\_history 0*

## D. Set Operations

Enter the following command to get all members of a set

**SMEMBERS** *user:1001:skills*

Enter the following command to check if member exists in set

**SISMEMBER** *user:1001:skills "Python"*

Enter the following command to get number of members

**SCARD** *user:1001:skills*

Enter the following command to find intersection of sets (common skills)

**SINTER** *user:1001:skills user:1002:skills*

Expected Output: {image}

## UPDATE Operations

### A. String Operations

Enter the following command to update existing value

```
SET user:1001:name "Alice Wanjiku"
```

Enter the following command to increment numeric value

```
SET user:1001:login_count 5
```

```
INCR user:1001:login_count
```

```
INCRBY user:1001:login_count 10
```

Enter the following command to decrement numeric value

```
DECR user:1001:login_count
```

```
DECRBY user:1001:login_count 2
```

Enter the following command to append to string value

```
SET user:1001:bio "Data Scientist"
```

```
APPEND user:1001:bio " | AI Enthusiast"
```

```
GET user:1001:bio
```

Expected Output: {image}

## **B. Hash Operations**

Enter the following command to update field in hash

```
HSET user:1002 age 33
```

Enter the following command to increment numeric field

```
HINCRBY user:1002 age 1
```

Enter the following command to update multiple fields

```
HMSET user:1002 role "senior_developer" department "Platform Engineering"
```

## **C. List Operations**

Enter the following command to update element at specific index

```
LSET user:1001:login_history 0 "2025-11-28 09:15:00 (Updated)"
```

## **D. Set Operations**

Enter the following command to add new members to set

```
SADD user:1001:skills "Kubernetes" "AWS"
```

Enter the following command to remove and add in one operation (not atomic, but sequential)

```
SREM user:1001:skills "Java"
```

```
SADD user:1001:skills "Golang"
```

## DELETE Operations

### A. Delete Keys

Enter the following command to delete a single key

```
DEL user:1001:created_at
```

Enter the following command to delete multiple keys

```
DEL user:1001:email user:1001:age
```

Enter the following command to delete key with expiration check and set a key with TTL

```
SET temp:key "temporary data" EX 10
```

```
TTL temp:key # Check time to live
```

(Wait 10 seconds and it auto-deletes)

Expected Output: {image}

### B. Hash Operations

Enter the following command to delete specific fields from hash

```
HDEL user:1002 department
```

Enter the following command to delete entire hash

```
DEL user:1002
```

### C. List Operations

Enter the following command to remove elements by value

```
LREM user:1001:notifications 1 "Welcome to the platform!"
```

Enter the following command to delete entire list

```
DEL user:1001:login_history
```

### D. Set Operations

Enter the following command to remove member from set

```
SREM user:1001:skills "Docker"
```

Enter the following command to remove random member

```
SPOP user:1001:skills
```

Enter the following command to delete entire set

```
DEL user:1001:skills
```

## 4. Applied Scenario: Real-Time Session Management System

### Problem Statement:

You're building a high-traffic web application (e.g., an e-learning platform) that needs to: Manage user sessions across multiple servers, Track active users in real-time, Implement rate limiting for API endpoints, Store temporary shopping cart data, Maintain real-time leaderboards for gamification

Traditional relational databases struggle with:

- ☐ High write throughput for session updates
- ☐ Sub-millisecond read latency requirements
- ☐ Automatic session expiration
- ☐ Horizontal scaling

### Solution: Redis Key-Value Store

Redis excels at this scenario because:

- ☐ In-memory storage provides microsecond latency
- ☐ Built-in TTL (Time To Live) for automatic session expiration
- ☐ Atomic operations prevent race conditions
- ☐ Horizontal scalability through Redis Cluster
- ☐ Support for complex data structures (hashes, sets, sorted sets)

### Implementation

Let's build a complete session management system step by step.

#### Step 1: User Authentication & Session Creation

When a user logs in, we create a session with a session id and store the session as a hash with 30-minute expiration. To avoid arguments errors, execute as one line command in redis cli:

```
HSET sess:550e8400-e29b-41d4-a716-446655440000 user_id "user:1001"  
username "alice@email.com" role "student" login_time "2025-11-28T10:30:00Z"  
ip_address "102.133.45.67" device "Chrome/MacOS"
```



Enter the following command to set session expiration (1800 secs = 30 min)

```
EXPIRE sess:550e8400-e29b-41d4-a716-446655440000 1800
```

Enter the following to add user to active users set (real-time tracking)

```
SADD active:users:2025-11-28 "user:1001"
```

Enter the following commands to track login event

```
RPUSH user:1001:login_events
```

```
"2025-11-28T10:30:00Z/102.133.45.67/Chrome/MacOS"
```

```
LTRIM user:1001:login_events -10 -1 # Keep last 10 login events
```

Enter the following command to increment daily login counter

```
INCR stats:logins:2025-11-28
```

Expected Output: {image}

## Step 2: Session Validation (Read)

In the real application, when user make a request, the session is validated by:

```
GET sess:550e8400-e29b-41d4-a716-446655440000 (do not run)
```

Since the session already exists, retrieve user details by running the following::

```
HGETALL sess:550e8400-e29b-41d4-a716-446655440000
```

Enter the following command to check remaining session time

```
TTL sess:550e8400-e29b-41d4-a716-446655440000
```

Enter the following command to get user's role for authorization

```
HGET sess:550e8400-e29b-41d4-a716-446655440000 role
```

Expected Output: {image}

## Step 3: Session Activity Update

After refreshing the session (EXPIRE sess:session\_id duration), enter the following command to update last activity timestamp

```
HSET sess:550e8400-e29b-41d4-a716-446655440000 last_activity  
"2025-11-28T10:45:00Z"
```

Enter the following command to track page views

```
INCR user:1001:page_views:2025-11-28
```

## Step 4: Shopping Cart Management

Enter the following command to add items to shopping cart (using hash)

```
HSET cart:user:1001 item:course101 "1/299.99/Introduction to Python"  
item:course205 "1/399.99/Advanced Data Structures" item:book42  
"2/29.99/Redis Essentials"
```

Enter the following command to set cart expiration (24 hours)

**EXPIRE** *cart:user:1001 86400*

Enter the following command to get cart contents

**HGETALL** *cart:user:1001*

Enter the following command to update item quantity

**HSET** *cart:user:1001 item:book42 "3/29.99/Redis Essentials"*

Enter the following command to remove item from cart

**HDEL** *cart:user:1001 item:course101*

Enter the following command to calculate cart total (application logic)

**HVALS** *cart:user:1001*

Expected Output: {image}

### Step 5: Rate Limiting (API Throttling)

Let's implement rate limiting: 100 requests per minute per user. We will use the following key format: *ratelimit:user\_id:minute*. Enter the following command to record request:

**INCR** *ratelimit:user:1001:2025-11-28:10:45*

**EXPIRE** *ratelimit:user:1001:2025-11-28:10:45 60*

Enter the following command to check if limit exceeded

**GET** *ratelimit:user:1001:2025-11-28:10:45*

Enter the following command to simulate multiple requests

**INCR** *ratelimit:user:1001:2025-11-28:10:45*

**INCR** *ratelimit:user:1001:2025-11-28:10:45*

**GET** *ratelimit:user:1001:2025-11-28:10:45*

### Rate Limiting Logic (Pseudocode):

```
```python
current_count = redis.get(f"ratelimit:user:{user_id}:{current_minute}")
if current_count and int(current_count) > 100:
    return "Rate limit exceeded. Try again later."
else:
    redis.incr(f"ratelimit:user:{user_id}:{current_minute}")
    redis.expire(f"ratelimit:user:{user_id}:{current_minute}", 60)
    # Process request
```
```

### Step 6: Real-Time Leaderboard (Sorted Sets)

We will now track student points using sorted sets.

The Score represents total points:

*ZADD leaderboard:november:2025 850 "user:1001" 920 "user:1002" 780 "user:1003" 1050 "user:1004" 890 "user:1005"*

Enter the following command to get top 3 students (highest scores)

*ZREVRANGE leaderboard:november:2025 0 2 WITHSCORES*

Enter the following command to get student's rank (0-indexed, from highest)

*ZREVRANK leaderboard:november:2025 "user:1001"*

Enter the following command to get student's score

*ZSCORE leaderboard:november:2025 "user:1001"*

Enter the following to increment student's score (earned 50 points)

*ZINCRBY leaderboard:november:2025 50 "user:1001"*

Enter the following command to get students within score range

*ZRANGEBYSCORE leaderboard:november:2025 800 900 WITHSCORES*

Enter the following command to get total number of students

*ZCARD leaderboard:november:2025*

Expected Output:{image}

### Step 7: Session Logout (Delete)

When the user logs out - delete session

*DEL sess:550e8400-e29b-41d4-a716-446655440000*

Enter the following command to remove from active users

*SREM active:users:2025-11-28 "user:1001"*

Optional: Enter the following to clear shopping cart if not persistent

*DEL cart:user:1001*

Enter the following commands to logout

*RPUSH user:1001:login\_events "2025-11-28T11:15:00Z/LOGOUT"*

*LTRIM user:1001:login\_events -10 -1*

### Step 8: Analytics Queries

# Get count of active users today

*SCARD active:users:2025-11-28*

# Get total logins today

*GET stats:logins:2025-11-28*

# Get user's page views today

*GET user:1001:page\_views:2025-11-28*

# Get recent login events for user

*LRANGE user:1001:login\_events 0 -1*

# Get all active sessions (pattern matching - use carefully!)

*KEYS sess:\**

### Scenario Performance Benefits:

#### Comparison: Redis vs Traditional RDBMS

| Operation         | Redis(Key-Value) | PostgreSQL(RDBMS)     |
|-------------------|------------------|-----------------------|
| Session Write     | < 1ms            | 5-20ms                |
| Session Read      | < 1ms            | 3-15ms                |
| Leaderboard Query | < 1ms            | 50-200ms              |
| Rate Limit Check  | < 1ms            | 10-30ms               |
| Auto-Expiration   | Native (TTL)     | Requires cleanup jobs |
| Concurrent Users  | Millions         | Thousands             |

#### Real-World Impact:

- ☐ Latency: 10-20x faster response times
- ☐ Throughput: Handle 100,000+ req/sec per instance
- ☐ Simplicity: No complex queries or indexes needed
- ☐ Cost: Lower server requirements due to efficiency

## 5. Advanced Operations

### Transactions (MULTI/EXEC)

We can achieve atomic operations -all or nothing- using transactions:

*MULTI*

*SET account:1001:balance 1000*

*DECRBY account:1001:balance 100*

*INCRBY account:1002:balance 100*

*EXEC*

### Pipelining (Batch Operations)

We can send multiple commands without waiting for responses (Reduce network round trips). In redis-cli, use the *--pipe* flag. In application code, use the pipeline feature. Eg: Enter the following command to create 5 users

## MULTI

```
SET user:2001 "User 2001"  
SET user:2002 "User 2002"  
SET user:2003 "User 2003"  
SET user:2003 "User 2004"  
SET user:2003 "User 2005"
```

## EXEC

### Pub/Sub (Real-Time Notifications)

Open 2 command prompts to simulate the following scenario. In Terminal 2, you will subscribe to notifications:

```
SUBSCRIBE user:1001:notifications
```

In Terminal 1, you will publish notification

```
PUBLISH user:1001:notifications "Your course has been approved!"
```

### Data Persistence Check

Enter the following command to save current state to disk

```
SAVE
```

Enter the following command to get last save time

```
LASTSAVE
```

Enter the following command to enable background save

```
BGSAVE
```

In order to check persistence configuration, use the following commands:

```
CONFIG GET save
```

```
CONFIG GET appendonly
```

### Performance Monitoring

Enter the following command to get server information

```
INFO
```

To Monitor commands in real-time, run the following in the 2nd terminal. Any prompts you run in the first terminal would be displayed here as logs

```
MONITOR
```

Enter the following command to get statistics, check the 2nd terminal for logs

```
INFO stats
```

Enter the following command to get memory usage, check 2nd terminal for any logs

*INFO memory*  
*MEMORY USAGE user:1001:login\_events*

## 6. Group Contribution Summary

### Optional: Remove Test Data

Enter the following command to delete all keys in the current database (BE CAREFUL!)

*FLUSHDB*

Or enter the following command to delete specific patterns

*DEL user:1001:name user:1001:bio cart:user:1001*

Enter the following command to exit Redis CLI

*EXIT*

### Stop Docker Container

Enter the following command to stop the container

*docker stop redis-lab*

### Key Takeaways

1. Use Meaningful Key Names: Follow naming conventions ( *object:id:field* )
2. Set Appropriate TTLs: Prevent memory bloat with expiration
3. Monitor Memory Usage: Use *INFO memory* regularly
4. Use Hashes for Objects: More memory efficient than multiple string keys
5. Implement Application-Level Validation: Redis doesn't enforce schemas

### Team Members and Responsibilities:

| Student ID | Member Name           | Contribution                              |
|------------|-----------------------|-------------------------------------------|
| 112721     | Kathembo Tsongo       | Setup Instructions, Docker Configuration, |
| 226022     | Kavira Neema<br>Nancy | CRUD operations                           |
| 225637     | Olang Sharon Leah     | Advanced Operations                       |
| 136371     | Joseph Vunanga        | Applied Scenario Implementation           |

# Appendix: Troubleshooting

## Common Issues:

### 1. Cannot connect to Redis:

Enter the following command to check if container is running

```
docker ps
```

Enter the following command to check logs

```
docker logs redis-lab
```

Enter the following command to restart container

```
docker restart redis-lab
```

### 2. Connection refused error:

Enter the following command to verify port mapping

```
docker port redis-lab
```

Enter the following command to check firewall settings on **Ubuntu**

```
sudo ufw status
```

### 3. Out of memory errors:

Enter the following command to check memory usage

```
INFO memory
```

Enter the following command to clear unnecessary data

```
FLUSHDB
```

Enter the following command to configure max memory

```
CONFIG SET maxmemory 256mb
```

```
CONFIG SET maxmemory-policy allkeys-lru
```

## References

### Further Learning Materials:

1. Official Documentation: <https://redis.io/docs/>
2. Redis University: Free courses at <https://university.redis.com/>
3. Redis Commands Reference: <https://redis.io/commands/>
4. Try Redis Online: <https://try.redis.io/>